Helwan Linux

Mastering PKGBUILD

From Beginner to Pro

Introduction: The Art of Packaging in Arch Linux

From Consumption to Craftsmanship

In the world of Linux, there are two types of users: those who settle for what the distribution provides, and those who refuse to be mere spectators—choosing instead to deconstruct and rebuild the system to suit their exact needs. If you are holding this book, you are undoubtedly the latter.

Arch Linux is more than just a distribution; it is a philosophy rooted in simplicity (**KISS**) and total control. At the heart of this philosophy lies the **Arch Build System (ABS)**. Learning to write a **PKGBUILD** is not merely a technical skill for organizing files; it is about "owning the keys to the system." it is the difference between waiting for a developer to update your favorite software and building it yourself from source in a matter of seconds.

Why this book?

This book is designed to be a practical roadmap, guiding you from absolute zero to professional mastery. We will not stop at explaining commands; we will dive deep into the "logic" that drives every line of code. You will learn how to respect your system and how to build clean packages that adhere to global Arch community standards.

What you will discover on this journey:

- **Building the Foundation:** Understanding the build philosophy and mastering Bash scripting.
- **The Package Lifecycle:** From the moment the source code is fetched to its final placement in system directories.
- **Managing Relationships:** How to make packages coexist and integrate through Dependencies, Conflicts, and Provides.
- **Advanced Mastery:** Handling Git sources and architecting complex Split Packages.
- **The Art of Debugging:** Detecting errors before they ever touch your system.

Who is this book for?

Whether you are a student taking your first steps into the world of **DevOps**, a developer looking to publish your software on the **AUR**, or an enthusiast who simply wants to understand what happens behind the scenes—this book is your companion.

Always remember: in Arch Linux, **you are the pilot**. This book is your flight manual to becoming a professional **Packager**.

Let the journey begin. The first line starts here.

Phase Zero: The Essential Foundation (If you're missing anything here, complete it first)

==========

0-1 Understanding the Arch Philosophy

What is Arch Linux?

The KISS Principle.

Why is there no auto-build system like Debian?

0-2 Bash Basics (Crucial) – You must understand:

Variables

Arrays

if / case

Functions

Exit codes

Quoting & Escaping

PKGBUILD = Bash Script

If you don't understand Bash, you will never master PKGBUILD.

What is Arch Linux?

Arch Linux is an independent Linux distribution focused on simplicity, full control, and bleeding-edge software (rolling release). Started in 2002, the Rolling Release model means there are no major annual version releases—updates are continuous. It is geared toward users who want to understand their system deeply. Installation is manual (no default GUI), and its most famous asset is the Arch Wiki (an incredibly powerful reference).

✨ The KISS Principle in Arch!

KISS = Keep It Simple, Stupid.

However, "Simple" here refers to simplicity in design, not necessarily being easy for beginners. The reason for this will be clear in the next section.

In Arch:

> **No tools to hide details from you.**
>
> **Minimal default configurations.**
>
> **No heavy modifications to packages.**
>
> **Everything stays as close to the original developer's intent as possible.**

This means the system is very "clean" and remains close to upstream.

🤔 Why is there no Auto Build System like Debian?

Let’s compare Arch with Debian:

🌑 Debian

Has a massive infrastructure.

Official Build Farm system.

Very strict policies.

Focuses on stability above all else.

Packages are sometimes modified to fit the Debian philosophy.

- **Arch Linux**

A completely different philosophy.

Uses a simple system called PKGBUILD.

Anyone can build a package themselves using makepkg.

No complex layers of bureaucracy or policy.

Relies on lighter and faster maintenance.

Arch aims for:

Speed + Simplicity + Reduced administrative complexity.

If they implemented a complex build system like Debian's:

It would increase bureaucracy.

It would slow down updates.

It would break the KISS principle.

🎁 **How are packages built then?**

The developer writes a PKGBUILD file, the package is built in a clean environment and then uploaded to the repositories. The user installs it using pacman. Additionally, there is:

AUR (Arch User Repository): A massive community sharing PKGBUILDs.

You can build almost anything yourself.

💻 Bash Basics (Abridged)

1. Variables

Bash

```
name="Ali"

echo $name
```

⚠️ No spaces around the = sign.

✅ Correct: x=5

❌ Wrong: x = 5

If the value contains spaces, use quotes: msg="Hello World"

2. Arrays

Bash

```
arr=("one" "two" "three")

echo ${arr[0]}     # one

echo ${arr[@]}     # All elements

echo ${#arr[@]}    # Number of elements
```

In a PKGBUILD, you will see formats like:

```
depends=("glibc" "gcc" "python")
```

3. if

Bash

```
if [[ $x -gt 5 ]]; then
   echo "Big"
else
   echo "Small"
fi
```

Important comparisons:

-eq : Equals

-ne : Does not equal

-gt : Greater than

-lt : Less than

4. case

A cleaner alternative to if:

Bash

```
case $1 in
   start)
```

```
        echo "Running"
        ;;
    stop)
        echo "Stopping"
        ;;
    *)
        echo "Unknown"
        ;;
esac
```

5. Functions

Bash

```
say_hi() {
    echo "Hello $1"
}
say_hi Ali
```

In a PKGBUILD, you will always see:

Bash

```
build() {
    make
}
package() {
    make install
}
```

6. Exit Codes (Very Important)

Every command returns a number:

0 = Success ✅

Any non-zero number = Error ❌

Example:

Bash

```
if make; then
   echo "Done"
else
   echo "There is a problem"
fi
```

You can use exit 1 to stop a script.

7. Quoting & Escaping (The most critical point)

Double quotes " ": Allows variable expansion.

name="Ali" -> echo "Hello $name" outputs Hello Ali.

Single quotes ' ': Prevents expansion.

echo 'Hello $name' outputs $name literally.

Escaping: echo "He said \"Hi\""

🌸 Why is this important for PKGBUILD?

Because PKGBUILD is 100% a Bash Script.

This means:

Variables = Package information (pkgname, pkgver).

Arrays = Dependencies.

Functions = build() / package().

Exit codes = Build failures.

Quoting = Prevents dangerous bugs.

==========

🟤 Phase 1: Your First PKGBUILD (The Simplest Form)

The goal of this stage isn't to "become a pro" or "understand everything."

🎯 The Only Goal:

Create a PKGBUILD.

Run makepkg.

Produce a working package.

Understand why every line exists.

1.1 What are we working on?

Not a complex program.

Not C.

Not Python.

☞ A single text file to be installed in /usr/bin.

This means:

No complex building.

No compiling.

No headaches.

1.2 Prepare the environment (Understand before you type)

Create an empty folder:

Bash

mkdir hello-pkg

cd hello-pkg

This folder will be:

Your build environment.

Your testing ground.

Completely safe; it poses no risk to your system.

1.3 The file we are packaging

Inside the folder, create a file named hello:

Bash

nano hello

Content:

Bash

#!/bin/bash

echo "Hello from my first PKGBUILD"

Then make it executable:

Bash

chmod +x hello

📌 Up to this point: This is just a Bash script that prints a sentence if you run ./hello.

1.4 Now: The PKGBUILD file itself

Create a file named PKGBUILD (Exactly like that, all caps):

Bash

nano PKGBUILD

Insert the following:

Bash

```
pkgname=hello-pkg
pkgver=1.0
pkgrel=1
pkgdesc="My first simple package"
arch=('any')
license=('MIT')
source=('hello')
sha256sums=('SKIP')

package() {
  install -Dm755 hello "$pkgdir/usr/bin/hello"
}
```

✋ **Stop here. We won't proceed until we understand every line.**

1.5 Understanding the lines one by one (Take your time)

pkgname

pkgname=hello-pkg

The package name as seen by pacman. It must be:

Lowercase.

No spaces.

pkgver

pkgver=1.0

The program version. In Arch, there is no forced format for this number.

pkgrel

pkgrel=1

The release version of the package. If you modify the PKGBUILD without changing the actual program → increment this number.

pkgdesc

pkgdesc="My first simple package"

A description for humans, not the machine.

arch

arch=('any')

Means: Not architecture-dependent (e.g., works on both x86_64 and ARM). It's just a text file.

license

license=('MIT')

We won't discuss licensing now, but it must be present.

source

source=('hello')

Tells makepkg: The source code is located locally.

sha256sums

sha256sums=('SKIP')

We are telling it: "Don't verify."

Why? Because we are beginners. We will learn the correct way later.

1.6 The most important part: package()

Bash

```
package() {
  install -Dm755 hello "$pkgdir/usr/bin/hello"
}
```

This literally means: Take the hello file and copy it to /usr/bin/hello, but inside the package directory, not the actual system.

📌 $pkgdir = A "fake" root directory.

Arch builds the package here safely before installation.

[Note]

The -D flag creates the directory path, and -m755 sets the executable permissions;

1.7 The Moment of Truth 🔥

Execute:

Bash

```
makepkg
```

If everything is correct, a .pkg.tar.zst file will appear.

Install it:

Bash

```
sudo pacman -U hello-pkg-1.0-1-any.pkg.tar.zst
```

Then run it:

Bash

```
hello
```

🎉 If the text appears, you have just created your first package!

1.8 Let this fact sink in:

Right now: You didn't just memorize a command, you didn't just copy from the AUR—you actually understand what every line does. This is the foundation of mastery.

==========

🌑 **Phase 2 — Understanding the True Depth (True Beginner)**

Goal of this phase:

❌ **NOT to increase complexity.**

❌ **NOT to build heavy software.**

✅ **TO UNDERSTAND:**

Why $pkgdir is vital.

Why touching the system directly is forbidden.

How a package is constructed step-by-step.

2.1 The most important concept in all of PKGBUILD

Understand this sentence well: A PKGBUILD does not install anything directly onto your system.

This means:

Not a single line writes to /usr.

Not a single line breaks the system.

2.2 So, where do the files go?

When you run makepkg, the Arch Linux build system does the following (in this order):

Creates a temporary directory.

Names it $pkgdir.

Tells you: "Do whatever you want… but only inside here."

This means when you write:

"$pkgdir/usr/bin/hello"

It actually points to something like:

/home/user/hello-pkg/pkg/hello-pkg/usr/bin/hello

📌 **NOT the real /usr/bin/hello.**

2.3 Why can't we copy directly to /usr? (A very important question)

Imagine a PKGBUILD contained: cp hello /usr/bin/hello

This would mean:

It modifies the system during the build process.

If it fails halfway → the system is contaminated.

pacman won't know where that file came from.

✕ This is a disaster. That is why Arch mandates: Everything goes to $pkgdir, and pacman is the only one allowed to touch the actual system.

2.4 Understanding the install command in depth

Let’s break down this line piece by piece:

install -Dm755 hello "$pkgdir/usr/bin/hello"

install: This is not for "installing packages"; it is a Unix command whose job is: Copying + Permissions + Directory Creation.

-D: Means "Create the directories if they don't exist." This ensures /usr and /usr/bin are created automatically.

-m755: Sets the permissions. It means:

Owner: Read + Write + Execute.

Group: Read + Execute.

Others: Read + Execute.

(This is the standard for executable programs).

2.5 Let's add something small (Practical Application)

We will add a documentation file.

Create a README file:

Bash

```
nano README
```

Content: This is my first Arch package.

Modify the PKGBUILD to look like this:

Bash

```
package() {
  install -Dm755 hello "$pkgdir/usr/bin/hello"
  install -Dm644 README "$pkgdir/usr/share/doc/hello-pkg/README"
}
```

Understanding the new line:

-m644: A regular file, not an executable.

Path: /usr/share/doc/hello-pkg/ (The standard location for documentation).

Build again:

Bash

makepkg -f

Then install:

Bash

sudo pacman -U hello-pkg-1.0-1-any.pkg.tar.zst

Check it:

Bash

cat /usr/share/doc/hello-pkg/README

2.6 A very important fact (Memorize this)

Any file not inside $pkgdir = Will not be part of the package.

This means:

Forgot to put it there? → It won't exist.

Put it in the wrong place? → It's lost.

Wrong path? → pacman won't see it.

2.7 Common Beginner Mistakes (Don't fall for these)

❌ **Using sudo inside a PKGBUILD.**

❌ **Writing to /usr directly.**

❌ **Forgetting $pkgdir.**

❌ **Using manual chmod instead of install -m.**

Solidify what you've learned before we move up. If you understand $pkgdir and install, you're ready.

==========

🌑 **Phase 3 — Source and Checksums (Logic, not Memorization)**

Goal of this phase — Understanding:

What "source" actually means.

Why Arch is strict about checksums.

How to fetch sources from:

Local files.

Archives.

Git.

...all without getting confused.

3.1 What is a "source"?

Let’s start with one sentence: Source is every file the program needs in order to be built.

It isn't necessarily code; for a large program, it could be: a single file, an archive, a script, or an entire repository.

3.2 What did we do previously?

In the last phase, we wrote:

source=('hello')

This meant:

The source exists in the same folder.

makepkg will not download anything.

It will simply perform an internal copy.

This is suitable only for learning, not for publishing.

3.3 Why do checksums even exist?

Arch tells you:

"I will build from source, but I want to be absolutely sure the source hasn't been tampered with."

Checksum = Fingerprint.

If the file changes—even by a single character:

→ The fingerprint changes.

→ The build fails.

📌 **This is security, not complexity.**

3.4 Why did we use SKIP before?

Because:

We were learning.

The file was local.

There was no downloading involved.

However:

❌ **This is not allowed in a real package.**

❌ **This is not allowed in the AUR.**

❌ **This is not allowed in a distribution.**

3.5 Let's do a real example (Archive)

Prepare a simple source and create a new folder:

Bash

mkdir hello-src

cd hello-src

Create a file:

Bash

nano hello.sh

Content:

Bash

#!/bin/bash

echo "Hello from archived source"

Then: chmod +x hello.sh

Archive it:

Bash

tar -czf hello-1.0.tar.gz hello.sh

3.6 PKGBUILD with a real source

Go back and create a new build folder:

Bash

cd ..

mkdir hello-archive-pkg

cd hello-archive-pkg

Copy the archive: cp ../hello-src/hello-1.0.tar.gz .

Create the PKGBUILD:

Bash

```
pkgname=hello-archive

pkgver=1.0

pkgrel=1

pkgdesc="Hello package built from tar.gz"

arch=('any')

license=('MIT')

source=('hello-1.0.tar.gz')

sha256sums=('PUT_CHECKSUM_HERE')


package() {

  install -Dm755 hello.sh "$pkgdir/usr/bin/hello-archive"

}
```

3.7 Getting the correct checksum (The non-manual way)

Execute:

Bash

makepkg -g

It will output a line like: sha256sums=('abc123...')

Copy it exactly and paste it in place of PUT_CHECKSUM_HERE.

📌 This is the correct way.

❌ We never calculate it manually.

3.8 What happened inside makepkg?

When you ran makepkg, the Arch system did the following:

Extracted the archive.

Placed the files in $srcdir.

Executed package().

Grabbed the files from $pkgdir.

Created the final package.

3.9 What is $srcdir?

Keep this sentence in mind: $srcdir = The location of the source after extraction.

If you have hello.sh, you will find it in $srcdir/hello.sh.

📌 This is why we write in package():

install -Dm755 hello.sh ... (because makepkg enters $srcdir by default).

3.10 A very common mistake

✕ Writing it like this:

install -Dm755 ../hello.sh ...

This is:

Broken.

Unreliable.

Dependent on your current directory.

Arch wants: $srcdir and $pkgdir, and nothing else.

3.11 Phase Summary (Crucial)

If you understand these, you have crossed the first professional threshold:

source = The source files.

sha256sums = Security.

makepkg -g = The correct method.

$srcdir ≠ $pkgdir

==========

🔘 Phase 4 — Git sources and pkgver() (Keep it Simple)

Goal of this phase — Understanding:

What a -git package actually is.

How makepkg handles Git.

Why pkgver() is essential.

How to create a working Git package without confusion.

4.1 What is a -git package?

Let's start with one sentence: A -git package = A package built from the latest code available in Git.

This means:

There is no fixed version (like 1.0 or 2.3).

Every build can be different.

It is suitable for development, not for absolute stability.

Because of this:

Its name must always end in -git.

It cannot have the same name as the stable version.

Example: foo (stable) vs. foo-git (Git version).

4.2 Practical Example (Very Simple)

We won't use a heavy project; we'll create a tiny local repo to understand the logic.

Create a simple Git repo:

Bash

```
mkdir hello-git-src
cd hello-git-src
git init
```

Create a file:

Bash

```
nano hello.sh
```

Content:

Bash

#!/bin/bash

echo "Hello from git package"

Then:

Bash

chmod +x hello.sh

git add hello.sh

git commit -m "first commit"

4.3 PKGBUILD for a Git Package

Go to another folder:

Bash

cd ..

mkdir hello-git-pkg

cd hello-git-pkg

Create the PKGBUILD:

Bash

pkgname=hello-git

pkgver=0

pkgrel=1

pkgdesc="Hello package built from git source"

arch=('any')

```
license=('MIT')

depends=('bash')

source=("git+file:///$(pwd)/../hello-git-src")

sha256sums=('SKIP')


pkgver() {

  cd "$srcdir/hello-git-src"

  git describe --long --tags 2>/dev/null || echo "r$(git rev-list --count HEAD).$(git rev-parse --short HEAD)"

}


package() {

  cd "$srcdir/hello-git-src"

  install -Dm755 hello.sh "$pkgdir/usr/bin/hello-git"

}
```

✋ **Stop here. Let's understand each part calmly.**

4.4 Understanding source with Git

```
source=("git+file:///...")
```

This means:

The source is a Git repository.

makepkg will perform: clone, update, and checkout.

📌 Any Git URL works: local, https, or ssh.

4.5 Why is sha256sums=('SKIP') allowed here? (Smart question! 🤔)

Because:

Git is constantly changing.

There is no fixed file to generate a static checksum for.

Arch allows SKIP only for Git (and other VCS) sources.

❌ It is not allowed for real tar.gz files or static files.

4.6 What is pkgver()?

Keep this sentence in mind: pkgver() determines the version number at the time of the build.

How it works:

makepkg calls this function.

It returns a string.

This string replaces the initial pkgver value.

The logic:git describe --long --tags

If there are tags → it uses them.

If not → it uses a fallback: r<commit-count>.<short-hash> (e.g., r3.a1b2c3).

This is a logical number that changes with every commit, allowing pacman to compare versions.

4.7 Why not leave pkgver static?

If you set pkgver=1.0 in a Git package, pacman will never know when there is an update. -Syu won't work correctly. This breaks the philosophy.

4.8 Practical Build

Execute:

Bash

makepkg

Then install:

Bash

sudo pacman -U hello-git-*.pkg.tar.zst

Run it:

Bash

hello-git

💪 **If the text appears, you're all set!**

4.9 Fatal Mistakes in Git Packages

❌ **Forgetting the pkgver() function.**

❌ **Using a static checksum with Git.**

❌ **Naming the package without the -git suffix.**

❌ **Relying on paths outside of $srcdir.**

4.10 Solidify the foundation

If you understand:

Why we use -git.

The source git+... syntax.

The role of pkgver().

The difference between static and Git versions.

Then you have officially moved from: 🥈 **Beginner** → Intermediate

==========

□ Phase 5 — Dependencies (The Professional Edge)

Goal of this phase — Understanding clearly:

- What a dependency actually is.
- The difference between:
  - `depends`
  - `makedepends`
  - `optdepends`
- When to place a package in each category.
- How to avoid bloating the user's system.

5.1 What is a dependency?

Let’s start from scratch:

Dependency = Something your program requires to either **run** or **build**.

- It’s not a luxury.
- It’s not a suggestion.
- It’s a necessity.

5.2 First Type: `depends`

```
depends=('bash')
```

It means:

- The program will **not run** without it.
- `pacman` will install it automatically.
- *Example:* If you have a Bash script, it needs `bash` at runtime → Put it in `depends`.
- □ **Common Mistake:** Putting something in `depends` when it’s not required at runtime.

5.3 Second Type: `makedepends`

```
makedepends=('gcc' 'make')
```

It means:

- Required **only** during the build process.
- The end-user does not need it to run the final program.
- *Example:* A C program needs `gcc` to be compiled. But once compiled, the executable runs on its own.

 Therefore: `gcc` → `makedepends`, NOT `depends`.

5.4 The Practical Difference (Very Important)

Item Requirement	Where to put it?
Required at Runtime	`depends`
Required only for Building	`makedepends`
Optional Features	`optdepends`

5.5 Third Type: `optdepends`

```
optdepends=('vim: edit configuration')
```

It means:

- The program works without it, but the experience is better with it.
- *Example:* A CLI tool that opens an editor. If `vim` exists → better experience. If not → the tool still functions.
- `pacman` shows these to the user but does **not** force their installation.

5.6 Modifying our previous example (Practical Application)

Let's modify our `hello-git` package. In the **PKGBUILD**:

Bash
```
depends=('bash')
optdepends=('figlet: fancy output')
```

Why?

- `bash` is essential.
- `figlet` is just for "fancy" aesthetics.

5.7 Common Fatal Mistakes

- Putting everything in `depends`.
- Forcing the user to install things they don't need.
- Bloating the system for no reason.

Arch Philosophy prefers:

- Minimum number of dependencies.
- Maximum user control.

5.8 Dependency does not mean "command"

This is a point where many people stumble.

- □ **Wrong:** `depends=('ls')`
- □ **Correct:** `ls` is part of `coreutils`. You would put `coreutils` if you truly needed it (which is rare).

5.9 How do I know where to put what?

Ask yourself this question:

"If I remove this package after installing my program, will the program still work?"

- **No** → `depends`
- **Yes** → `makedepends` or `optdepends`

5.10 Mental Exercise (No coding required)

You have a program written in **Python**, it is built using **setuptools**, and it produces a script that runs via **python**.

Where do you put:

1. `python` → ?
2. `setuptools` → ?

The Answer:

1. `python` → `depends`
2. `setuptools` → `makedepends`

If you answered that instinctively, you've got it.

5.11 Phase Summary

You now understand:

- Dependency placement is not random.
- Arch is strict to keep the system "clean."

- Your choice of placement = Respect for the user.

  This is what distinguishes a professional from many others in the AUR.

□ Phase 6 — prepare() / build() / check() - (Why are there multiple functions?)

Goal of this phase — Understanding:

- The role of each function.
- When to use them.
- The true execution order.
- How to write a clean PKGBUILD that anyone after you can understand.

6.1 The True Execution Order in Arch

Memorize this sequence as logic, not just names:

1. **`prepare()`**
2. **`build()`**
3. **`check()`**
4. **`package()`**

□ **None of these are mandatory (except `package()`).** □ **But each has a distinct and clear purpose.**

6.2 `prepare()` — Preparation Before Anything Else

Simply put: **"Prepare the source before building."** Use it for:

- Patching files.
- Applying patches (`patch -p1 ...`).
- Modifying permissions.
- Preparing configurations.

Simple Example:

Bash
```
prepare(){
  cd "$srcdir/project"
  sed -i 's/DEBUG=1/DEBUG=0/' config.mk
}
```

□ **Note:** We are not building here; we are only modifying the source. □ **Common Mistake:** Putting build commands inside `prepare()`.

6.3 `build()` — The Actual Construction

It means: **"Transform the source into a ready-to-run program."** This is where you place:

- `make`
- `cmake`
- `meson`
- `python setup.py build`
- Any compilation commands.

Simple Example:

Bash
```
build() {
  cd "$srcdir/project"
  make
}
```

 Note: `build()` does **not** write to the system and does **not** copy final files. Its only job is to compile/build.

6.4 `check()` — Testing (Optional)

It means: **"Ensure that what was built works correctly."** Use it if: The project includes tests.

- `make test`
- `pytest`
- `ctest`

Example:

Bash
```
check() {
  cd "$srcdir/project"
  make test
}
```

 If tests fail, `makepkg` will stop. **This is a good thing, not a bad thing.**

6.5 `package()` — Final Packaging (The Most Important)

It means: **"Copy the final output to `$pkgdir`."** Only here should you use:

- `install`
- `cp`
- Setting final permissions.
- Copying docs and configs.

 Forbidden in `package()`: Building, patching, or downloading.

6.6 A Complete Example (Structured and Clear)

Let’s look at an organized PKGBUILD:

Bash
```
pkgname=hello-structured
pkgver=1.0
pkgrel=1
pkgdesc="Structured example"
arch=('any')
license=('MIT')
depends=('bash')
source=('hello.sh')
sha256sums=('SKIP')

prepare() {
  chmod +x hello.sh
}

build() {
  echo "Nothing to build"
}

package() {
  install -Dm755 hello.sh "$pkgdir/usr/bin/hello-structured"
}
```

 Here:

- `prepare` → Setup.
- `build` → No build (completely normal for scripts).
- `package` → Packaging.

6.7 A Very Common Mistake

 Putting everything inside `package()`. Why is this wrong?

- Hard to read.
- Hard to debug.
- Violates Arch philosophy. **Arch prefers:** Each stage in its designated place.

6.8 When should I leave a function empty?

Actually, you shouldn't.

- No `prepare`? → Remove the function.
- No `build`? → Remove the function.
- No `check`? → Remove the function. □ **Do not leave them empty in the script for no reason.**

6.9 Mental Exercise (Important)

If you have a program that needs a **patch**, then **compilation**, then **installation**: **The correct order is:**

1. Patch → `prepare()`
2. Compile → `build()`
3. Install → `package()`

If you answered that automatically → **You are on the right track.**

6.10 Phase Summary

Now you:

- Understand the package lifecycle.
- Can read any project's PKGBUILD.
- Know whether a line is in its correct place or not.

□ Phase 7 — Split Packages - (One Package → Multiple Packages)

Goal of this phase — Understanding:

- What a split package actually is.
- Why Arch uses this method.
- How to write it correctly.
- How to avoid confusing both `pacman` and the user.

7.1 What is a Split Package?

Let's start with the golden rule: **Split package = One source generating multiple packages.** This means:

- **One** build process.
- **One** PKGBUILD file.
- **But the output:** A CLI package, a GUI package, a docs package, and a libs package.

7.2 Why do we do this?

Because:

- The user should only install what they actually need.
- It keeps the system lightweight.
- It reduces unnecessary dependencies.
- *Real-world example:* A program has a library, a command-line tool, and a GUI. It makes no sense to force a GUI installation on a headless server.

7.3 The Basic Structure of a Split Package

Instead of: `pkgname=foo` We write: `pkgname=(foo foo-cli foo-docs)`

 Every name here: Is an independent package with its own files and its own distinct dependencies.

7.4 Essential Structure: `pkgbase` vs `pkgname`

- **`pkgbase`** = The name of the overall project.
- **`pkgname`** = The names of the resulting packages.

Example:

Bash
```
pkgbase=hello
pkgname=(hello-cli hello-docs)
```

7.5 `package_*()` — The True Heart

Every package gets its own specific function:

Bash
```
package_hello-cli() {
  install -Dm755 hello.sh "$pkgdir/usr/bin/hello"
}

package_hello-docs() {
  install -Dm644 README "$pkgdir/usr/share/doc/hello/README"
}
```

 Note: `$pkgdir` changes for each package automatically. You don't need to create separate directories manually.

7.6 A Complete Example (Crystal Clear)

Bash

```
pkgbase=hello
pkgname=(hello-cli hello-docs)
pkgver=1.0
pkgrel=1
pkgdesc="Hello split package example"
arch=('any')
license=('MIT')
source=('hello.sh' 'README')
sha256sums=('SKIP' 'SKIP')

package_hello-cli() {
  depends=('bash')
  install -Dm755 hello.sh "$pkgdir/usr/bin/hello"
}

package_hello-docs() {
  # No dependencies needed for docs
  install -Dm644 README "$pkgdir/usr/share/doc/hello/README"
}
```

The result:

1. `hello-cli.pkg.tar.zst`
2. `hello-docs.pkg.tar.zst`

7.7 Dependencies in Split Packages

This is where many people make mistakes.

- **Wrong:** Putting `depends=('bash')` globally if only the CLI needs it.
- **Correct:** Defining `depends` inside the specific `package_` function if it only applies to that part.

Note: Documentation packages usually have zero dependencies.

7.8 Fatal Mistakes

- Putting all files into a single `package()` function.
- Forgetting to split them logically.
- Making a "docs" package depend on a "GUI" package.

The goal of split packages is control, not complexity.

7.9 When should you use Split Packages?

Use them if:

- The project is large and has multiple distinct components.
- The user likely doesn't need everything at once.

 Don't use them: Just because "it looks professional" or if the project is tiny.

7.10 Mental Exercise

A program contains: a **binary**, a **man page**, and a **README**. How would you split it? **Answer:**

- `foo` → binary.
- `foo-docs` → README and man page. *(Optionally, `foo-man` if the man pages are very large, but usually they go with docs).* If you reached this conclusion on your own → **Excellent.**

7.11 Phase Summary

Now you:

- Understand split packages.
- Can read complex PKGBUILDs.
- Won't be confused when you see `pkgname=()`. This is a level of proficiency that many never reach.

---

8 Phase 8 — .install Files and Hooks (When to use them and when it's a crime)

Goal of this phase — Understanding:

- What a `.install` file actually is.
- When you truly need it.
- The difference between it and a PKGBUILD.
- The most dangerous mistakes that occur because of it.

8.1 What is a `.install` file?

Let's start with a clear sentence: **A `.install` file = A script that executes during installation or removal, NOT during the build.**

- Not during `makepkg`.
- **During `pacman` execution only.**

8.2 PKGBUILD vs .install (Fundamental Difference)

Feature	PKGBUILD	.install

Feature	PKGBUILD	.install
Execution Time	Build Time	Installation/Removal Time
Touches the System?	□ No	□ Yes
Uses sudo?	□ No	□ No (`pacman` is already root)

□ **Important:**

`.install` runs as **root**.

Any mistake = **Danger**.

8.3 When do we actually need `.install`?

Use it **only** if you need to:

- Update caches (icons / desktop databases).
- Reload systemd daemons.
- Display a message to the user.
- Enable a service (use with extreme caution).

□ **Do NOT use it for:**

- Copying files.
- Building software.
- Modifying default configurations.

8.4 Structure of a `.install` file

The name should always be: `pkgname.install` (e.g., `hello.install`).

8.5 Available Functions within it

- `post_install()` → After the first-time installation.
- `post_upgrade()` → After a version update.
- `post_remove()` → After the package is removed.

8.6 A Simple and Safe Example

Let's create a file: `nano hello.install`

Content:

Bash
```
post_install() {
  echo "Thank you for installing hello"
}

post_remove() {
  echo "Goodbye from hello"
}
```

 Note: This is safe; it does not modify the system, it just displays a message.

8.7 Linking `.install` to PKGBUILD

In your **PKGBUILD**, you must specify that an install file exists:

```
install=hello.install
```

Example:

Bash
```
pkgname=hello
pkgver=1.0
pkgrel=1
install=hello.install
```

Without this line, the file will be ignored and won't execute.

8.8 Important Practical Example (systemd service)

Real-world scenario: A program adds a service.

 Very Wrong:

`systemctl enable foo.service` (inside the script).

Why?

- The user hasn't consented.
- Direct system interference.
- Against the Arch philosophy.

 The Correct Way:

Bash
```
post_install() {
  echo "You can enable the service with:"
  echo "  systemctl enable foo.service"
}
```

 Always: Inform the user — do not force settings.

8.9 Fatal Mistakes (Never do these)

- Deleting files not owned by the package in `post_remove()`.
- Modifying `/etc` files automatically.
- Running network commands.
- Enabling services without permission.

If you see a PKGBUILD doing this: **It is a poorly made package**, even if it "works."

8.10 Hooks vs .install (Important Distinction)

- **.install:** Specific to the package, simple, and limited in scope.
- **Hooks:** System-wide, complex, and usually not for beginners.

 As a Beginner/Intermediate: Focus on `.install` and leave hooks for now.

8.11 Phase Summary

Now you understand:

- `.install` is dangerous if used incorrectly.
- PKGBUILD builds; `.install` installs.
- Respecting the user is more important than "convenience."

Phase 9 — conflicts / provides / replaces

Goal of this phase — Understanding:

- Why a package might conflict with another.
- How to let `pacman` recognize alternatives.
- How to write these fields in a safe and professional manner.

9.1 What does `conflicts` mean?

The golden rule: **`conflicts` = Packages that cannot be installed alongside this package at the same time.**

Practical Example:

Bash
```
pkgname=foo
conflicts=('foo-old')
```

The meaning: If a user has `foo-old` installed, `pacman` will prevent the installation or suggest removing the old one first.

9.2 When should you use `conflicts`?

- When two packages contain the exact same files (same paths).
- When a new version completely supersedes an older, differently named package.

 Common Mistake: Writing random `conflicts` entries. This prevents the installation of unrelated packages.

9.3 What does `provides` mean?

The golden rule: **`provides` = This package acts as a substitute for a default package or an interface.**

Example:

Bash
```
pkgname=python-pypy
provides=('python')
```

The meaning: Any program requiring `python` can run using `python-pypy`. `pacman` will recognize this as a valid alternative solution.

9.4 When should you use `provides`?

- When a package is an alternative to another.
- When different projects provide the same service.
- *Practical Example:* `foo-cli` vs `foo-gui`. Both might provide the same backend library; you can add `provides=('foo-lib')` to both.

9.5 What does `replaces` mean?

The golden rule: **`replaces` = This package will take the place of a specific package during installation.**

Example:

Bash

```
pkgname=foo-new
replaces=('foo-old')
```

The meaning: When you install `foo-new`, `pacman` will automatically remove `foo-old` without triggering an error or a conflict.

9.6 A Complete Practical Example

Bash
```
pkgname=foo
pkgver=1.0
pkgrel=1
pkgdesc="Example of conflicts/provides/replaces"
arch=('any')
license=('MIT')
conflicts=('foo-old')
provides=('foo-lib')
replaces=('foo-old')

package() {
  install -Dm755 hello.sh "$pkgdir/usr/bin/hello"
}
```

The result:

- Protects the system from conflicts.
- `pacman` knows exactly what to remove and what to keep.

9.7 Common Fatal Mistakes

- □ Adding `conflicts` for unrelated packages → prevents the user from installing their software.
- □ Using `replaces` without full understanding → might delete important files.
- □ Using `provides` incorrectly → confuses `pacman` during dependency resolution.

9.8 Phase Summary

Now you know:

- **`conflicts`** → Prevents simultaneous installation with the old package.
- **`provides`** → Offers an "alternative interface."
- **`replaces`** → Directly substitutes the old package.

□ Phase 10 — Debugging & Common Mistakes

Goal of this phase:

- Learn how to discover why `makepkg` failed.
- Avoid the mistakes that most beginners fall into.
- Create a package that works on the first try, or at least learn how to fix it quickly.

10.1 The Most Important Step: Read the output carefully

When you run `makepkg`, follow these steps:

1. Read the messages that appear in order.
2. Look for keywords like: `error`, `failed`, `cannot find`.
3. Focus on the specific file or line that caused the issue.

Pro-tip: Never ignore a **Warning**. Sometimes warnings reveal future problems.

10.2 Common Mistakes in PKGBUILD

Mistake	Reason	Solution
Forgetting `$pkgdir`	Files go to the actual system	Use `install -D` and `$pkgdir`
Using `sudo`	Modifying the system during build	Don't use `sudo` — `pacman` handles root
Forgetting checksums	Security failure	Use `makepkg -g` to get the `sha256`
Everything in `package()`	Mixing preparation and building	Use `prepare()`, `build()`, and `package()` correctly
Missing dependencies	Program fails after installation	Define `depends` and `makedepends` wisely
Files outside `$srcdir`	`makepkg` fails to find them	Copy files to `$srcdir` first
Wrong `replaces` / `conflicts`	`pacman` deletes/blocks packages	Verify the relationships accurately
Bad `.install` root mods	Potential to break the system	Use only for messages or safe system hooks

10.3 How to handle Checksum failures

- If you are using **Git** → set it to `'SKIP'`.
- If you are using **tar.gz** → **Do NOT** use `'SKIP'`.
- Run: `makepkg -g`
- Copy the resulting fingerprint into `sha256sums`.

 Note: Checksums protect users from corrupted or tampered-with files.

10.4 Debugging "Command Not Found"

If `makepkg` says: `command not found`

The Solution: Add the program to `makedepends` or ensure it is installed on your build system.

10.5 Practical tips to reduce errors

- Start with a small and simple PKGBUILD.
- Test every stage: `prepare()`, `build()`, `check()`, and `package()`.
- Use `makepkg -f` to quickly retry builds.
- Don't rush into split packages or `.install` until you've mastered the basics.
- Read PKGBUILDs from popular **AUR** packages as examples.

10.6 The Professional Verification Method

Run: `makepkg -s --nocheck`

- `-s` → Automatically installs dependencies.
- `--nocheck` → Skips the `check()` function if you want a quick trial.

Once the build succeeds, run the full process including `check()`. This significantly reduces the friction of the first trial.

10.7 Phase Summary

If you have mastered:

- Reading and understanding errors.
- Using the correct flags.
- Knowing where each function belongs in the PKGBUILD.

Then you have reached a professional level in Arch Linux packaging.

⍰ Phase 11 — Final Professional Tips

Goal of this phase:

- Make any PKGBUILD you write readable, secure, and clean.
- Learn best practices before publishing a package to the AUR or your own distribution.
- Avoid the common pitfalls encountered by beginners and new professionals alike.

11.1 Clear Comments = Gold

For every stage, dependency, or function: write a comment explaining **why** it is there. *Example:*

Bash
```
# This source is required because the program needs hello.sh to run
source=('hello.sh')
sha256sums=('SKIP')
```

Reason: Anyone reading your PKGBUILD a year from now will understand everything without guessing.

11.2 Use prepare / build / check / package Intelligently

- Do not mix their functions.
- Keeping every stage in its place makes **debugging** much easier.
- A well-structured PKGBUILD allows any beginner to understand it within a minute.

11.3 Split Packages + Git Sources

- **Git:** Always use `pkgver()` and `'SKIP'` for checksums.
- **Split Packages:** Use specific `package_*()` functions for each separate package.
- **Dependencies:** Ensure each sub-package manages its own specific `depends`. **Result:** A clean, modular PKGBUILD ready for production.

11.4 Dependencies = Mental Organization

- **Runtime** → `depends`
- **Build time** → `makedepends`
- **Optional** → `optdepends`
- In split packages, define dependencies within the specific package functions to maintain precision.

11.5 .install = A Tool, Not a Toy

- Use it only for notifications or safe system changes.
- Avoid touching the system unless absolutely necessary.
- Informational messages for the user are always better than executing automatic commands.

11.6 Naming Matters

- **Git packages:** Must end with the `-git` suffix.
- **Split packages:** Use clear, unified, and consistent naming conventions.
- The AUR and Arch community value naming clarity and professional standards.

11.7 Test Your Package Before Publishing

- `makepkg -s` → To test automatic dependency installation.
- `makepkg -f` → To test rebuilding from scratch.
- Run the `check()` function to verify integrity.
- Ensure all files in `$pkgdir` are in their correct intended paths.
- Test both **installation** and **removal**.

 This is an essential step before any submission to the AUR or the **Helwan Linux** repository.

11.8 Additional Tips

- Study the PKGBUILDs of famous packages as examples.
- Keep your PKGBUILDs version-controlled using **Git**.
- Test your packages in a **Virtual Machine** before installing them on your main system.
- **Never** use `sudo` inside a PKGBUILD.

11.9 Curriculum Summary

Now, you officially know:

1. Every core variable (`pkgname`, `pkgver`, `pkgrel`, `source`, `sha256sums`).
2. The full lifecycle (`prepare`, `build`, `check`, `package`).
3. Dependency types and their proper usage.
4. How to handle **Split Packages** and **Git Sources**.
5. How to use `.install` files safely.
6. How to manage `conflicts`, `provides`, and `replaces`.
7. How to **Debug** and avoid common mistakes.